NANYANG TECHNOLOGICAL UNIVERSITY

COLLEGE OF COMPUTING AND DATA SCIENCE (CCDS)



SC2006: SOFTWARE ENGINEERING

LAB#4: IMPLEMENTATION, TESTING, AND PREPARATION FOR DEMO DELIVERABLES

AUTHOR: UVADHARANYA RAMESH (U2522415G)

1. **Black Box Testing**
   I. Scoring service
   II. Equivalence Class and Boundary Value Testing
   III. Test Cases and Results

2. **White Box Testing**
   I. **EligibilityEvaluator._percentile**
      a. Control Flow Graph
      b. Basic Path Testing
      c. Test Cases and Results
   II. **ALevelRP90Strategy.calculate**
      a. Control Flow Graph
      b. Basic Path Testing
      c. Test Cases and Results

3. **Coding agent exercises**
   I. Regenerating Class
   II. Generating UML Diagram

1. Black Box Testing

I. Control class under test: Scoring service

The scoring service (scoring/services.py) is the control class that implements the admission score logic (use cases: Calculate Admission Score and Match Eligible Courses; functional requirement 3). It was chosen for black box testing because it has clear inputs, clear outputs and well-defined limits.

The service has two parts.

Score calculation. The user's qualification type and grades are passed in, and the matching strategy is chosen automatically. Three schemes are supported:

- A-Level, 70-point scheme (2025 cohort onwards): the best three H2 grades plus General Paper (counted at H1 weight). A fourth content subject, and H1 Mother Tongue, are only included if they raise the score; the result is rebased to 70.
- A-Level, 90-point scheme (2024 cohort and earlier): three H2 grades, one more content subject, General Paper and Project Work. H1 Mother Tongue is included only if it raises the score.
- Polytechnic: a cumulative GPA from 0.00 to 4.00, rounded to two decimal places.

Invalid input is rejected with a message (for example "Please enter a valid grade for all required subjects." or "GPA must be between 0.00 and 4.00"). Any input not given for a compulsory field is also rejected with a message ("Please fill in this field.")

Eligibility evaluation. The calculated score is compared with a course's published admission data, either the 10th and 90th percentile cut-offs or the offer rate for the score band. Each course is given a likelihood category (Very likely, Likely, Borderline, Unlikely or No data). Only Very likely and Likely courses are treated as eligible.

Information managed by the service: qualification type, H2/H1 grades, General Paper, Mother Tongue, Project Work, GPA, and the course cut-offs and offer rates.

II. Equivalence Class and Boundary Value Testing

Equivalence class testing. The input domain is split into classes whose members the system should treat in the same way. One representative from each valid class (handled normally) and each invalid class (rejected with an error) is tested, and invalid classes are tested one at a time so that a failure can be traced to its cause.

Boundary value testing. For inputs with an order (GPA, number of subjects, scores and offer rates), the values on and just either side of each limit are also tested, since errors tend to occur at the edges.

1. calculate_score: A-Level

Input	Valid equivalence classes	Invalid equivalence classes	Boundary values tested
H2 grades	Three or more grades from A,	-	best score AAA and

Input	Valid equivalence classes	Invalid equivalence classes	Boundary values tested
	B, C, D, E, S, U (any case)		worst score UUU
General Paper	One grade from the set	-	-
H1 (4th subject)	Not given; one grade from the set	-	4th subject that gives exactly the same score as without it
Mother Tongue	Not given; one grade from the set	A grade outside the set	-
Project Work (90 scheme)	One grade from A, B, C, D, E, S, U	A grade outside the set, Invalid input ("x"), no input	-
H1 and Project Work (90 scheme)	Both given with valid grades	4th subject missing; Project Work missing	Best score and worst score

2. calculate_score: Polytechnic GPA

Input	Valid equivalence classes	Invalid equivalence classes	Boundary values tested
GPA	A number from 0.00 to 4.00	Below 0; above 4.00; not a number; empty	-0.01, 0.00, 4.00, 4.01; rounding at 3.755

3. Eligibility evaluation

For the percentile data, the test course has a 10th percentile of 65.00 and a 90th percentile of 70.00 (A-Level), or 3.50 and 3.80 (Polytechnic GPA). A score is Borderline if it is below the 10th percentile by no more than 2.5 rank points (A-Level) or 0.10 GPA (Polytechnic).

Output class	A-Level score	Polytechnic GPA	Boundary values tested
Very likely	70.00 or more	3.80 or more	70.00 / 69.99; 3.80
Likely	65.00 up to below 70.00	3.50 up to below 3.80	65.00 / 64.99; 3.50
Borderline	62.50 up to below 65.00	3.40 up to below 3.50	62.50 / 62.49; 3.40 / 3.39
Unlikely	Below 62.50	Below 3.40	62.49; 3.39

For offer-rate data: 70% or more is Very likely, 40% up to below 70% is Likely, 15% up to below 40% is Borderline, below 15% is Unlikely, and no data gives No data. Boundaries tested: 70 / 69.99, 40 / 39.99, 15 / 14.99.

Note: Dropdown fields such as H2 grades and H1 grades prevents invalid inputs, so we focused only on the cases a user can actually produce.

III. Test Cases and Results

The test cases were automated with pytest (scoring/test_blackbox.py). In total, 40 test cases were executed and all 40 passed.

a) calculate_score: A-Level (70-point scheme)

Input parameters: H2 grades, General Paper (GP), H1 subject, Mother Tongue (MTL), Project Work (PW)

No.	Test Input	Expected Output	Actual Output	Pass?
A-1	(Valid) H2 “AAA”, GP “A”	70.00	70.00	Yes
A-2	(Valid) H2 “UUU”, GP “U”	0.00	0.00	Yes
A-3	(Valid) H2 “ABC”, GP “B”	61.25	61.25	Yes
A-4	(Valid) H2 “AAA”, GP “C”, H1 “A”	67.81 (4th subject counted)	67.81 (4th subject counted)	Yes
A-5	(Valid) H2 “ABC”, GP “B”, H1 “E”	61.25 (4th subject not counted)	61.25 (4th subject not counted)	Yes
A-6	(Boundary) H2 “AAA”, GP “A”, H1 “A”	70.00 (4th subject not counted)	70.00 (4th subject not counted)	Yes
A-7	(Valid) H2 “AAA”, GP “B”, H1 “A”, MTL “A”	69.03 (4th subject counted)	69.03 (4th subject counted)	Yes
A-8	(Valid) H2 “CABA”, GP “A”	67.50	67.50	Yes
A-9	(Valid) H2 “aaa”, GP “a”	70.00	70.00	Yes
A-10	(Invalid) H2 “AAA”, GP “A”, MTL “Z”	ERROR: Please enter a valid grade for all required subjects.	ERROR: Please enter a valid grade for all required subjects.	Yes

b) calculate_score: A-Level (90-point scheme)

No.	Test Input	Expected Output	Actual Output	Pass?
B-1	(Valid) H2 “AAA”, GP “A”, H1 “A”, PW “A”	90.00 (4th subject counted)	90.00 (4th subject counted)	Yes
B-2	(Valid) H2 “UUU”, GP “U”, H1 “U”, PW “U”	0.00 (4th subject counted)	0.00 (4th subject counted)	Yes

No.	Test Input	Expected Output	Actual Output	Pass?
B-3	(Valid) H2 “ABB”, GP “B”, H1 “C”, PW “A”	81.25 (4th subject counted)	81.25 (4th subject counted)	Yes
B-4	(Invalid) H2 “ABB”, GP “B”, PW “A” (no 4th subject)	ERROR: Please enter a valid grade for all required subjects.	ERROR: Please enter a valid grade for all required subjects.	Yes
B-5	(Invalid) H2 “ABB”, GP “B”, H1 “C” (no Project Work)	ERROR: Please enter a valid grade for all required subjects	ERROR: Please enter a valid grade for all required subjects	Yes
B-6	(Invalid) H2 “ABB”, GP “B”, H1 “C”, PW “x”	ERROR: Please enter a valid grade for all required subjects.	ERROR: Please enter a valid grade for all required subjects.	Yes

c) calculate_score: Polytechnic GPA

Input parameter: GPA

No.	Test Input	Expected Output	Actual Output	Pass?
C-1	(Invalid) GPA “-0.01”	ERROR: GPA must be between 0.00 and 4.00	ERROR: GPA must be between 0.00 and 4.00	Yes
C-2	(Valid) GPA “0.00”	0.00	0.00	Yes
C-3	(Valid) GPA “4.00”	4.00	4.00	Yes
C-4	(Invalid) GPA “4.01”	ERROR: GPA must be between 0.00 and 4.00	ERROR: GPA must be between 0.00 and 4.00	Yes
C-5	(Valid) GPA “3.755”	3.76	3.76	Yes
C-6	(Invalid) GPA “” (no input given)	ERROR: GPA must be between 0.00 and 4.00	ERROR: GPA must be between 0.00 and 4.00	Yes

d) Eligibility evaluation

Input parameters: score, course admission data. Test course: A-Level 10th percentile 65.00 and 90th percentile 70.00; Polytechnic GPA 10th percentile 3.50 and 90th percentile 3.80.

A-Level score against percentile cut-offs

No.	Test Input	Expected Output	Actual Output	Pass?
E-1	(Boundary) A-Level score	Very likely; eligible=True	Very likely;	Yes

No.	Test Input	Expected Output	Actual Output	Pass?
	70.00		eligible=True	
E-2	(Boundary) A-Level score 69.99	Likely; eligible=True	Likely; eligible=True	Yes
E-3	(Boundary) A-Level score 65.00	Likely; eligible=True	Likely; eligible=True	Yes
E-4	(Boundary) A-Level score 64.99	Borderline; eligible=False	Borderline; eligible=False	Yes
E-5	(Boundary) A-Level score 62.50	Borderline; eligible=False	Borderline; eligible=False	Yes
E-6	(Boundary) A-Level score 62.49	Unlikely; eligible=False	Unlikely; eligible=False	Yes

Polytechnic GPA against percentile cut-offs

No.	Test Input	Expected Output	Actual Output	Pass?
E-7	(Boundary) GPA 3.80	Very likely; eligible=True	Very likely; eligible=True	Yes
E-8	(Boundary) GPA 3.50	Likely; eligible=True	Likely; eligible=True	Yes
E-9	(Boundary) GPA 3.40	Borderline; eligible=False	Borderline; eligible=False	Yes
E-10	(Boundary) GPA 3.39	Unlikely; eligible=False	Unlikely; eligible=False	Yes

Offer-rate categories and missing data

No.	Test Input	Expected Output	Actual Output	Pass?
E-11	(Boundary) offer rate 70	Very likely	Very likely	Yes
E-12	(Boundary) offer rate 69.99	Likely	Likely	Yes
E-13	(Boundary) offer rate 40	Likely	Likely	Yes
E-14	(Boundary) offer rate 39.99	Borderline	Borderline	Yes
E-15	(Boundary) offer rate 15	Borderline	Borderline	Yes
E-16	(Boundary) offer rate 14.99	Unlikely	Unlikely	Yes
E-17	(Valid) offer rate not available	No data	No data	Yes

No.	Test Input	Expected Output	Actual Output	Pass?
E-18	(Valid) A-Level score 70.00, course with no admission data	No data; eligible=False	No data; eligible=False	Yes

2. White Box Testing

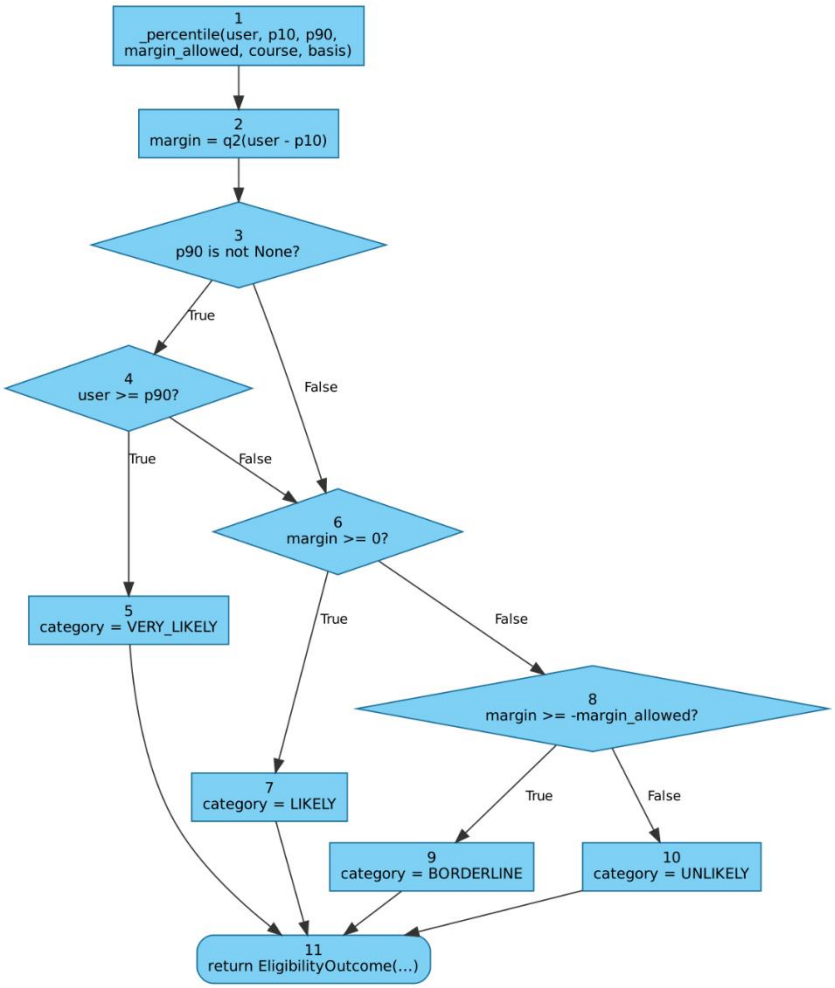
White box tests are designed from the code itself using basis path testing. For each method, the control flow graph (CFG) is drawn, the cyclomatic complexity is calculated (decision points + 1, which equals edges − nodes + 2), a set of independent paths is chosen, and one test case is written for each path. Both methods below are in scoring/services.py and belong to the scoring service tested in Section 1.

I. EligibilityEvaluator._percentile

_percentile decides how likely a student is to be admitted to a course, based on the course's published 10th and 90th percentile cut-offs (UC9). It receives the student's score, the 10th percentile (p10), the 90th percentile (p90, which may be missing) and the size of the borderline margin. It returns an outcome with a likelihood category (Very likely, Likely, Borderline or Unlikely), whether the course counts as eligible, and the margin above or below the 10th percentile.

I.I Control Flow Graph

Figure 1: Control flow graph of _percentile



[image is uploaded separately if it isn't clear]

The condition p90 is not None and user >= p90 is a compound condition, so each part is counted as a separate decision point (nodes 3 and 4).

I.II Basis Path Testing

Cyclomatic Complexity = | decision points | + 1 = 4 + 1 = 5 (decision points: nodes 3, 4, 6 and 8).

Basis Paths (the baseline is the longest path; each other path changes one decision from the baseline):

Baseline path: 1, 2, 3, 4, 6, 8, 10, 11

Basis path 2 (node 3 is False): 1, 2, 3, 6, 8, 10, 11

Basis path 3 (node 4 is True): 1, 2, 3, 4, 5, 11

Basis path 4 (node 6 is True): 1, 2, 3, 4, 6, 7, 11

Basis path 5 (node 8 is True): 1, 2, 3, 4, 6, 8, 9, 11

All five paths can be executed.

I.III Test Cases and Results

_percentile(user, p10, p90, margin_allowed), with margin_allowed = 2.5 (the A-Level borderline margin in rank points).

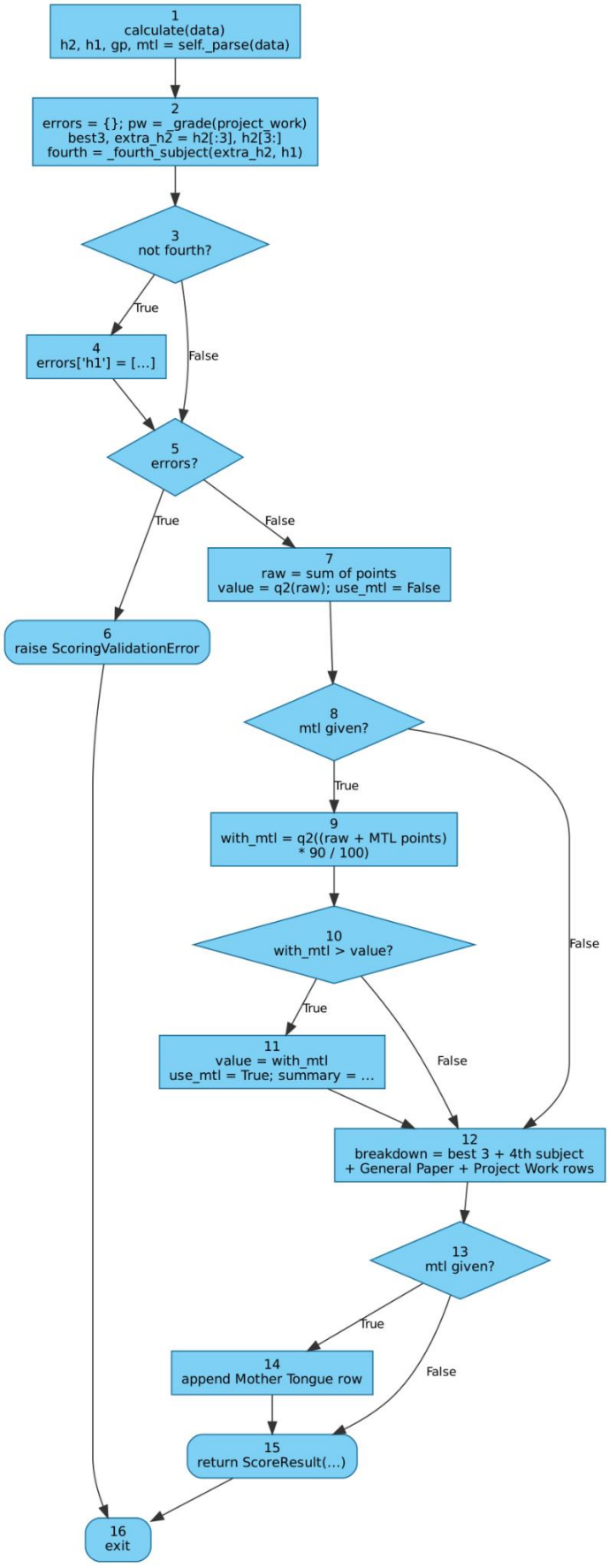
No.	Path	Test Input	Expected Output	Actual Output	Pass?
WB1-1	1 (baseline)	user = 60.00, p10 = 65.00, p90 = 70.00	Unlikely; eligible=False; margin=-5.00	Unlikely; eligible=False; margin=-5.00	Yes
WB1-2	2	user = 60.00, p10 = 65.00, p90 = None	Unlikely; eligible=False; margin=-5.00	Unlikely; eligible=False; margin=-5.00	Yes
WB1-3	3	user = 72.00, p10 = 65.00, p90 = 70.00	Very likely; eligible=True; margin=7.00	Very likely; eligible=True; margin=7.00	Yes
WB1-4	4	user = 67.00, p10 = 65.00, p90 = 70.00	Likely; eligible=True; margin=2.00	Likely; eligible=True; margin=2.00	Yes
WB1-5	5	user = 63.50, p10 = 65.00, p90 = 70.00	Borderline; eligible=False; margin=-1.50	Borderline; eligible=False; margin=-1.50	Yes

II. ALevelRP90Strategy.calculate

ALevelRP90Strategy.calculate computes the A-Level score for the older 90-point scheme (2024 cohort and earlier). It reads and checks the subject grades, rejects input that is missing a fourth content subject or a valid Project Work grade, adds up the points for three H2 subjects, the fourth subject, General Paper and Project Work, and then includes H1 Mother Tongue (rebased from 100 to 90) only if that raises the score. It returns the score with a breakdown of which subjects were counted.

II.I Control Flow Graph

Figure 2: Control flow graph of ALevelRP90Strategy.calculate


1
calculate(data)
h2, h1, gp, mtl = self._parse(data)
2
errors = {}; pw = _grade(project_work)
best3, extra_h2 = h2[:3], h2[3:]
fourth = _fourth_subject(extra_h2, h1)
3
not fourth?
True
False
4
errors['h1'] = [...]
5
errors?
True
False
6
raise ScoringValidationError
7
raw = sum of points
value = q2(raw); use_mtl = False
8
mtl given?
True
False
9
with_mtl = q2((raw + MTL points)
* 90 / 100)
10
with_mtl > value?
True
False
11
value = with_mtl
use_mtl = True; summary = ...
12
breakdown = best 3 + 4th subject
+ General Paper + Project Work rows
13
mtl given?
True
False
14
append Mother Tongue row
15
return ScoreResult(...)
16
exit

[image is uploaded separately if it isn't clear]

II.II Basis Path Testing

Cyclomatic Complexity = | decision points | + 1 = 5 + 1 = 6 (decision points: nodes 3, 5, 8, 10 and 13).

Basis Paths:

Baseline path: 1, 2, 3, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16

Basis path 2 (node 3 is True): 1, 2, 3, 4, 5, 6, 16

Basis path 3 (node 5 is True): 1, 2, 3, 5, 6, 16

Basis path 4 (node 8 is False): 1, 2, 3, 5, 7, 8, 12, 13, 15, 16

Basis path 5 (node 10 is False): 1, 2, 3, 5, 7, 8, 9, 10, 12, 13, 14, 15, 16

Basis path 6 (node 13 is False): 1, 2, 3, 5, 7, 8, 9, 10, 11, 12, 13, 15, 16 — infeasible

Path 6 cannot be executed. Nodes 8 and 13 test the same condition (if mtl), and mtl does not change between them, so it is not possible for node 8 to be True and node 13 False. Therefore, only five independent paths can be tested, and five test cases are written.

II.III Test Cases and Results

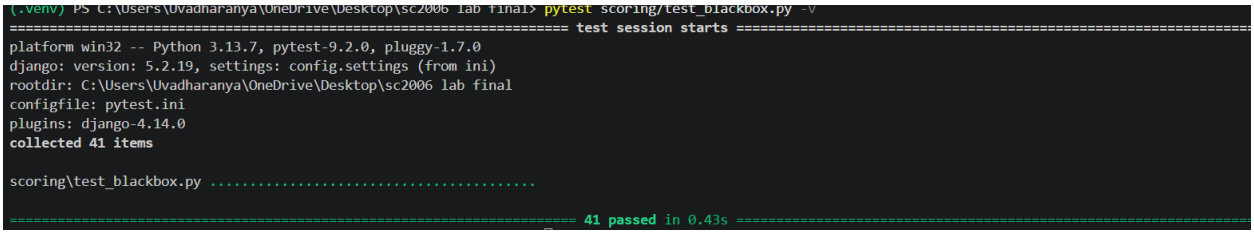
ALevelRP90Strategy.calculate(data), where the data holds the H2 grades, General Paper (GP), H1 subject, Project Work (PW) and Mother Tongue (MTL)

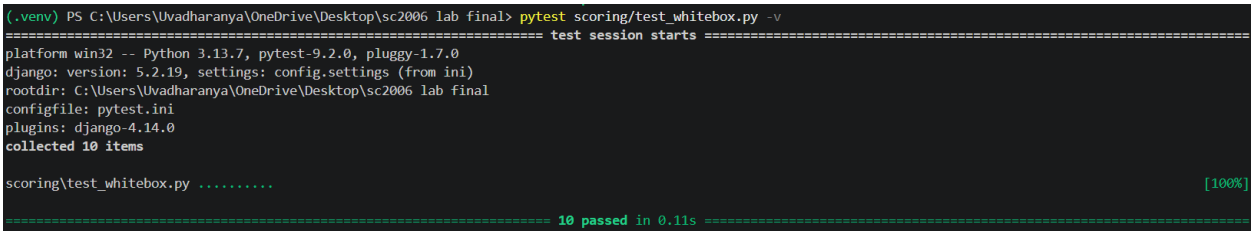
No.	Path	Test Input	Expected Output	Actual Output	Pass?
WB2-1	1 (baseline)	H2 "ABB", GP "B", H1 "C", PW "A", MTL "A"	82.13; MTL counted	82.13; MTL counted	Yes
WB2-2	2	H2 "ABB", GP "B", PW "A", no H1, no MTL	ERROR: Please enter a valid grade for all required subjects.	ERROR: Please enter a valid grade for all required subjects.	Yes
WB2-3	3	H2 "ABB", GP "B", H1 "C", PW ""	ERROR: Please enter a valid grade for all required subjects.	ERROR: Please enter a valid grade for all required subjects.	Yes
WB2-4	4	H2 "ABB", GP "B", H1 "C", PW "A", no MTL	81.25; no MTL	81.25; no MTL	Yes
WB2-5	5	H2 "ABB", GP "B", H1 "C", PW "A", MTL "E"	81.25; MTL not counted	81.25; MTL not counted	Yes

Worked calculations: without Mother Tongue the score is 20 + 17.5 + 17.5 (H2) + 7.5 (H1 C) + 8.75 (GP B at H1 weight) + 10 (PW A) = 81.25. In WB2-1, adding MTL “A” gives (81.25 + 10) × 90 / 100 = 82.125, which rounds to 82.13 and is higher, so it is counted. In WB2-5, MTL “E” gives (81.25 + 5) × 90 / 100 = 77.63, which is lower, so it is not counted.

In total, 10 white box test cases were executed (scoring/test_whitebox.py) and all 10 passed.

Results from running test_whitebox.py and test_blackbox.py:





3. Coding agent exercises

Exercise 3.3.2: Regenerating a control class from scratch

Class chosen and preparation

We used the scoring service (scoring/services.py), the control class that we black box tested. It calculates admission scores for A-Level (70-point and 90-point schemes) and Polytechnic GPA, and works out how likely a student is to be admitted to each course (UC8 and UC9).

Before deleting anything, we wrote our test cases: 40 black box tests (equivalence classes and boundary values, test_blackbox.py) and 10 white box tests (basis paths for two methods, test_whitebox.py). All of them passed against our original code. We then deleted the whole of services.py.

Tool and materials provided

We used the AI chat agent in VS Code (GitHub Copilot). We gave it our class diagram, black box and white box test files, the rest of the codebase, views.py, models. We asked it to recreate services.py so that the tests pass.

Prompting and iterations

First prompt: Our file scoring/services.py has been deleted. Recreate it so that all tests in scoring/tests.py, scoring/test_blackbox.py and scoring/test_whitebox.py pass. Read those tests, scoring/views.py, the models and the class diagram in the file. Do not modify any other files.

The agent produced a complete file from this single prompt. All 10 white box tests passed but only 39 of the 40 black box tests passed. The two failures were:

- **A-8 (four H2 subjects):** with H2 grades "CABA" and General Paper "A", we expected 67.50 but the generated code returned 72.19, overstating the student's score, which matters because it makes a student look more eligible than they are.
- **B-2 (90-point scheme, all grades U):** the score was correct (0.00), but the generated code also added a Project Work warning, so the agent added behaviour we had not specified.

Second prompt: The services.py file you have created, passes all the whitebox test cases but only passed 38 out of the 40 blackbox testcases. These are the two failed cases: A-8 and B-2. Please review the code once again and try to fix the error. You can also read any other necessary files like controllers.py, serializers.py if needed. Please also mention the change you've made.

After this one extra round, all 40 black box tests and all 10 white box tests passed. In total the exercise needed two prompts and no extended conversation.

Outcome and evaluation

Overall, the results met our expectations. The AI agent generated a complete working class that mostly passes all of the test cases just within two prompts. It also correctly used the internal method names used by our white box tests, showing that it referred and understood the implementation of the existing files.

This exercise also showed the limitations of using only test cases as the instructions. The agent made assumptions where requirements were unclear, such as including unrequested warnings. The incorrect fourth H2 subject weightage, taking 15 points instead of 7.5, created a small error that wouldn't be obvious from just the user interface. This shows that even though the generated code passed all the tests, we shouldn't blindly trust it because tests don't catch everything. Hence, we kept our original code in the real project.

To improve, we could have given the AI chat more context, including the scoring rules, existing tests along with their expected output and code comments. We could also have asked the agent to state the assumptions it was making and add more test cases to ensure all the possible scenarios were taken into considerations and carefully review it before adding it into the project.

Exercise 3.3.3: Generating the updated class diagram

We chose to use the AI agent to generate an updated UML class diagram for our UniFinder project. The diagram was generated based on the existing Django codebase, covering the main entities, control classes, their attributes and methods, and the relationships between them.

Tool and materials provided

We used the AI chat agent in VS Code (GitHub Copilot) and provided it with our UniFinder source code, particularly the Django apps' models.py, services.py and views.py files. We asked it to generate a UML class diagram in Mermaid format, focusing on the key entity and control classes, their main attributes and methods, and their relationships.

Prompting and iterations:

First prompt:

Read the apps in this repository (accounts, universities, courses, comparison, scoring, scholarships, guidance, careers, community, recommender), mainly models.py, services.py and views.py. Produce a UML class diagram in Mermaid (classDiagram) of the key classes only (about 25 at most).

Include: entity classes with their attributes, control classes with their public methods, relationships with multiplicities (associations, inheritance, composition), stereotypes: <<entity>> or <<control>>.

Do not modify any files. After the diagram, list any assumptions you made

Second prompt:
Your diagram is missing the classes Comment (community/models.py), EligibilityEvaluator, ScoringService, and SystemInitializer. Add them with their main members and relationships. Also fix the Mermaid syntax so stereotypes are written inside the class braces (e.g., <<entity>>) inside the class body. Don't change anything else

After this additional round, the agent reported that it had corrected the Mermaid syntax and added the missing classes.

Outcome and evaluation

The agent generated an initial diagram that included the main entities and six controllers as well as the scoring strategies. It also identified several attributes and relationships that weren't present in our original Lab3 diagram, reflecting changes made during implementation. This showed us where the design deviated, which feeds directly into the traceability section of Lab 5.

However, we also noticed several important classes were omitted. These include: EligibilityEvaluator, ScoringService and SystemInitializer. The generated Mermaid code also has a syntax issue which could prevent the diagram from being created correctly. Furthermore, the diagram was too messy, making it harder to read it as a whole.

In our second prompt, we named the missing classes and the syntax problem. The agent then corrected everything mentioned, providing a new diagram and Mermaid code. We had to also verify that the UML diagram matched with the given source code.

Overall, the AI did partially meet our expectations. The first draft saved a lot of time, but it still required human review and correction. Something we could have done differently is asking for the diagram in smaller parts to keep it readable and check it step by step confirming correctness.